

Automate

Limite de temps : 2 secondes

Limite de mémoire : 1024 Mio

Considérons une ligne constituée de N cases, numérotées 0 à $N - 1$ de gauche à droite. Chaque case a une hauteur distincte : la hauteur de la case i est p_i , et la séquence $p_0, p_1, \dots, p_{N-1}$ est une permutation des nombres $0, 1, \dots, N - 1$.

Deux cases ayant pour **indices** i et j sont dites *proches* si et seulement si $|i - j| \leq 1$. En particulier, chaque case est proche d'elle-même.

Un robot se trouve sur la ligne, placé initialement sur une certaine case. Le robot accepte des commandes, chacune de l'un des deux types suivants :

- **MAX** : parmi toutes les cases proches de la case actuelle du robot, la prochaine position du robot sera la case unique de hauteur **maximale** ;
- **MIN** : parmi toutes les cases proches de la case actuelle du robot, la prochaine position du robot sera la case unique de hauteur **minimale**.

Puisqu'une cellule est proche d'elle-même, une commande peut laisser le robot sur la même case.

Un *programme* est une suite finie de commandes, où chaque commande est soit **MAX**, soit **MIN**. Si le robot démarre à la cellule X et suit le programme S , il termine à une cellule déterminée de manière unique, notée $\text{result}(S, X)$.

On vous donne Q requêtes, chacune est un ensemble de cases de départ de taille K_i ($0 \leq i < Q$). Plus précisément, la i -ème requête concernera certaines cases $x_0, x_1, \dots, x_{K_i-1}$; le robot sera placé sur l'une d'elles, mais on ne sait pas à l'avance laquelle. Pour chaque requête, vous devez déterminer s'il existe un programme qui amène le robot à la même case finale, peu importe la position de départ parmi x qui sera utilisée.

Formellement, vous devez déterminer s'il existe un programme S tel que :

$$\text{result}(S, x_0) = \text{result}(S, x_1) = \dots = \text{result}(S, x_{K_i-1}).$$

La permutation p est **la même** pour toutes les requêtes.

Notez que vous n'avez pas besoin de construire un tel programme – seulement d'indiquer s'il en existe un.

Détails d'implémentation

La première fonction que vous devez implémenter est :

```
void initialize(std::vector<int> p)
```

- p : une permutation des nombres de 0 à $N - 1$.

La deuxième fonction que vous devez implémenter est :

```
bool exists_program(std::vector<int> x)
```

- x : un tableau spécifiant les cases dans un ordre strictement croissant pour une requête.

La fonction `initialize` est appelée exactement une fois, avant tout appel à la fonction `exists_program`.

La fonction `exists_program` est appelée Q fois, une fois pour chaque requête. Elle doit renvoyer `true` s'il existe un programme avec lequel le robot se retrouve sur la même case peu importe de laquelle des cases données il est parti, et `false` sinon.

Contraintes

- $3 \leq N \leq 2 \cdot 10^5$
- $1 \leq Q \leq 5 \cdot 10^5$
- $p_0, p_1, \dots, p_{N-1}$ est une permutation des nombres $0, 1, \dots, N - 1$
- $2 \leq K_i \leq N$
- $0 \leq x_0 < x_1 < \dots < x_{K_i-1} < N$ pour chaque requête.
- La somme des K_i pour les Q requêtes ne dépasse pas 10^6 .

Exemples

Entrée de l'évaluateur d'exemple 1	Sortie de l'évaluateur d'exemple 1
3 3 0 2 1 2 0 2 3 0 1 2 2 1 2	111

Entrée de l'évaluateur d'exemple 2	Sortie de l'évaluateur d'exemple 2
7 3	001
0 4 2 1 3 5 6	
3 0 1 3	
3 1 3 4	
2 3 6	

Explication de l'exemple 1

Pour cet exemple, $p = [0, 2, 1]$. Pour la deuxième requête, le robot peut commencer à la case 0, à la case 1 ou à la case 2. Considérons le programme `[MAX]`.

- Lorsque le robot commence à la case 0 : les cases proches de 0 sont les cases d'indices 0 et 1. Lorsque le robot exécute la commande `MAX`, puisque $p_0 < p_1$, il se déplace vers la case 1.
- Lorsque le robot commence à la case 1 : les cases proches de 1 sont les cases d'indices 0, 1 et 2. Lorsque le robot exécute la commande `MAX`, puisque $p_0 < p_1$ et $p_1 > p_2$, il reste à la case 1.
- Lorsque le robot commence à la case 2 : les cases proches de 2 sont les cases d'indices 1 et 2. Lorsque le robot exécute la commande `MAX`, puisque $p_1 > p_2$, il se déplace vers la case 1.

Il existe donc un programme après lequel le robot se retrouve à la case 1 à partir des trois cases de départ, et la réponse pour cette requête est `true`. D'autres programmes fonctionnent aussi, comme `[MIN, MAX]`, `[MIN, MIN, MAX, MIN, MAX, MIN]` et `[MAX, MIN, MAX]`.

Explication de l'exemple 2

Pour cet exemple, $p = [0, 4, 2, 1, 3, 5, 6]$.

Pour les deux premières requêtes, on peut montrer qu'il n'existe aucun programme pour lequel le robot se retrouve dans la même case à partir de chaque case de départ donnée, donc la réponse aux deux est `false`.

Considérons la dernière requête, pour laquelle le robot peut commencer à la case 3 ou à la case 6, et considérons le programme `[MAX, MAX, MAX]`.

- Quand le robot commence à la case 3 : les cases proches de 3 sont les cases d'indices 2, 3, et 4. Puisque $p_3 < p_4$ et $p_2 < p_4$, le robot se déplace vers la case 4. En suivant les deux commandes suivantes de la même manière, il se retrouve à la case 6.
- Quand le robot commence à la case 6, il reste à la case 6 tout au long et se retrouve là.

La réponse à cette requête est donc `true`. Le programme `[MIN, MAX, MAX, MAX]` fonctionne aussi. Cependant, d'autres programmes peuvent ne pas fonctionner. Par exemple, si le programme est `[MAX, MAX]`, le robot se retrouve à la case 5 quand il commence à la case 3, mais il se retrouve à la case 6 quand il commence à la case 6.

Sous-tâches

Sous-tâche	Points	N	Q	K_i	Contraintes supplémentaires
0	0	-	-	-	Exemples du sujet.
1	3	$\leq 2 \cdot 10^5$	$\leq 5 \cdot 10^5$	$= 2$	Si la réponse à une requête est positive, il existe un programme valide qui utilise exactement 1 commande.
2	7				Si la réponse à une requête est positive, il existe un programme valide qui utilise au plus 5 commandes.
3	9	≤ 100	≤ 500	$= 2$	-
4	17	$\leq 5\,000$	$\leq 5 \cdot 10^5$	$= 2$	-
5	7	$\leq 2 \cdot 10^5$			$x_1 = x_0 + 1$ pour chaque requête.
6	8	$\leq 5\,000$	$\leq 5 \cdot 10^5$	$\leq N$	-
7	13	$\leq 2 \cdot 10^5$	$\leq 5 \cdot 10^5$	$\leq N$	La permutation est d'abord croissante, puis décroissante, puis croissante. Formellement, il existe 2 entiers a, b tels que $0 < a < b < N - 1$ et $p_0 < p_1 < \dots < p_a > p_{a+1} > \dots$ $> p_b < p_{b+1} < \dots < p_{N-1}$.
8	13				$p_0 < p_1 > p_2 < p_3 > \dots < p_{N-1}$ si N est pair, ou $p_0 < p_1 > p_2 < p_3 > \dots > p_{N-1}$ si N est impair.
9	23				-

Évaluateur d'exemple

Le format d'entrée est le suivant :

- ligne 1 : deux entiers – les valeurs de N et Q ;
- ligne 2 : N entiers $p_0, p_1, \dots, p_{N-1}$, où p_i est la hauteur de la i -ème case ;
- ligne $3 + i$: les cases données pour la i -ème requête – un entier K_i , suivi sur la même ligne de K_i entiers $x_0, x_1, \dots, x_{K_i-1}$, les cellules de départ possibles du robot.

Le format de sortie est le suivant :

- ligne 1 : une chaîne binaire de longueur Q , dont le i -ème caractère est 1 si la réponse à la i -ème requête est `true`, et 0 sinon.